

Звіт. Гордій Андрій IT-31. Практична №5

Завдання 1:

Було створено функцію `print_card()` без параметрів та викликано її тричі. Після цього створено функцію `print_card_args()` з параметрами та виконано її виклики позиційними, іменованими та змішаними аргументами.

Для параметра `group` задано значення за замовчуванням "IT-31". Також було перевірено помилкові виклики функції.

При виклику `print_card_args("Ivan")` виникає помилка:

TypeError: print_card_args() missing 3 required positional arguments: 'surname', 'group', and 'year'

Це відбувається через те, що передано недостатню кількість аргументів.

При виклику `print_card_args(name="Ivan", "Petrenko")` виникає:

SyntaxError: positional argument follows keyword argument

Ця помилка виникає під час запуску програми, оскільки позиційний аргумент не можна ставити після іменованого.

Код:

```
def print_card():
    print("Name: Andriy Hordiy")
    print("Group: IT-31")
    print("Birth Year: 2009")
print_card()
print_card()
print_card()

def print_card_args(name, surname, group, year):
    print(name, surname, group, year)
print_card_args("Andriy", "Hordiy", "IT-31", "2009")
print_card_args(surname="Hordiy", name="Andriy", year="2009", group="IT-31")
print_card_args("Andriy", "Hordiy", year="2009", group="IT-31")

def print_card_args2(name, surname, year, group="IT-31"):
    print(name, surname, group, year, "Виведено з групою за замовчуванням!")
print_card_args2("Andriy", "Hordiy", "2009")

# print_card_args("Ivan")
# print_card_args(name="Ivan", "Petrenko")
```

Результат:

```
/Users/admin/Desktop/ПРОГРАМУВАННЯ/.venv/bin/python /Users/admin/Desktop/ПРОГРАМУВАННЯ/Практична 5/task1.py
Name: Andriy Hordiy
Group: IT-31
Birth Year: 2009
Name: Andriy Hordiy
Group: IT-31
Birth Year: 2009
Name: Andriy Hordiy
Group: IT-31
Birth Year: 2009
Andriy Hordiy IT-31 2009
Andriy Hordiy IT-31 2009
Andriy Hordiy IT-31 2009
Andriy Hordiy IT-31 2009 Виведено з групою за замовчуванням!
```

Завдання 2:

Спочатку було створено функцію `print_age(year)`, яка за допомогою `print()` виводить вік. Функція не має `return`, тому нічого не повертає. Друга функція `get_age(year, current_year=2026)` за допомогою `return` повертає вік. Параметр `current_year` має значення за замовчуванням 2026.

Було викликано обидві функції. При виконанні `print(print_age(y))` спочатку виводиться вік, а потім `None`. `None` з'являється тому, що `print_age()` не має `return`, а функція без `return` автоматично повертає `None`.

Результат `get_age()` було використано для обчислення віку в місяцях і тижнях. Для цього отриманий вік помножено відповідно на 12 та 52. Також через параметр `current_year` було визначено вік у 2030 році.

Після цього було перевірено використання `print_age(y) * 12`. Виникає помилка `TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'`, тому що `print_age()` повертає `None`, а `None` не можна помножити на число.

До `get_age()` було додано перевірку: якщо рік народження більший за поточний рік або від'ємний, функція одразу повертає -1. Перевірка на значенні 3000 показала результат -1.

Після `return` було додано `print("after return")`. Цей рядок не виконується, оскільки `return` одразу завершує роботу функції.

Код:

```
y = 20027
def print_age(year):
```

```

    print("Ваш вік:", 2026 - year)
def get_age(year, current_year=2026):
    if year > current_year or year < 0:
        return -1

    return current_year - year
    print("after return")

print_age(y)
print(get_age(y))
print(print_age(y))

age = get_age(y)
print("Вік в місяцях:", age * 12)
print("Вік в тижнях:", age * 52)
age_2030 = get_age(y, 2030)
print("У 2030 році мені буде:", age_2030)
# print_age(y) * 12

```

Результат:

```

/Users/admin/Desktop/ПРОГРАМУВАННЯ/.venv/bin/python /Users/admin/Desktop/ПРОГРАМУВАННЯ/Практична 5/task2.py
Ваш вік: 17
17
Ваш вік: 17
None
Вік в місяцях: 204
Вік в тижнях: 884
У 2030 році мені буде: 21

```

Завдання 3:

Було створено функції для отримання ініціалів, підрахунку букв, голосних та перевертання рядка. Для всіх функцій задано анотації типів.

У програмі виведено повне ім'я Andriy Hordiy, ініціали А.Н., довжину прізвища, кількість голосних і приголосних. Також у циклі визначено кількість кожної голосної букви в прізвищі.

Було перевірено параметр letter зі значенням за замовчуванням. При виклику count_letters(surname) використовується буква "а", тому результат дорівнює 0.

Прізвище було перевернуто за допомогою циклу та отримано yidroH.

Наприкінці виведено `count_letters.__doc__`, який показав `None`, оскільки для функції не було задано `docstring`. За допомогою `count_letters.__annotations__` отримано анотації параметрів `text`, `letter` та типу, який повертає функція.

Код:

```
name = "Andriy"
surname = "Hordiy"

def get_initials(name: str, surname: str) -> str:
    return name[0] + "." + surname[0] + "."

def count_letters(text: str, letter: str = "a") -> int:
    count = 0
    for char in text:
        if char == letter:
            count += 1
    return count

def count_vowels(text: str) -> int:
    vowels = "aeiouy"
    count = 0

    for char in text:
        if char in vowels:
            count += 1
    return count

def reverse_text(text: str) -> str:
    result = ""
    for char in text:
        result = char + result
    return result

print("Повне ім'я:", name, surname)
print("Ініціали:", get_initials(name, surname))

c = len(surname)
vowels = count_vowels(surname)
consonants = c - vowels

print("Довжина прізвища:", c)
print("Голосних:", vowels)
print("Приголосних:", consonants)
for vowel in "aeiou":
    print(vowel, ":", count_letters(surname, letter=vowel))

print("count_letters(surname):", count_letters(surname))

print("Перевернуте прізвище:", reverse_text(surname))

print(count_letters.__doc__)
print(count_letters.__annotations__)
```

Результат:

```
/Users/admin/Desktop/ПРОГРАМУВАННЯ/.venv/bin/python /Users/admin/Desktop/ПРОГРАМУВАННЯ/Практична 5/task3.py
Повне ім'я: Andriy Hordiy
Ініціали: A.H.
Довжина прізвища: 6
Голосних: 3
Приголосних: 3
a : 0
e : 0
i : 1
o : 1
u : 0
count_letters(surname): 0
Перевернуте прізвище: ydnoH
None
{'text': <class 'str'>, 'letter': <class 'str'>, 'return': <class 'int'>}
```

Завдання 4:

Створено функцію `read_grade(prompt)`, яка запитує оцінку від 0 до 100. Для перевірки введення використовується метод `isdigit()`. Якщо введено неправильне значення, програма виводить повідомлення про помилку та повторює запит.

Створено функцію `to_letter(grade)`, яка перетворює числову оцінку на буквену за заданою шкалою. Для перевірки використано декілька умов `if` та `return` без `elif`.

Створено функцію `average(grades)`, яка обчислює середнє арифметичне оцінок.

Створено функцію `count_above(grades, limit)`, яка підраховує кількість оцінок, що перевищують задане значення.

Створено функцію `print_report(name, group, grades)`, яка виводить звіт: список оцінок, середнє значення, буквений результат, найкращий і найгірший бал та кількість оцінок вище середнього.

Створено функцію `main()`, у якій задаються ім'я та група студента. Кількість оцінок визначається за кількістю букв в імені. Для введення оцінок використовується цикл `for`.

У кінці програми виконано єдиний виклик функції `main()`.

Код:

```
def read_grade(prompt):
    while True:
        grade = input(prompt)
```

```

        if grade.isdigit():
            grade = int(grade)

            if 0 <= grade <= 100:
                return grade

    print("Введіть ціле число від 0 до 100.")

def to_letter(grade):
    if grade >= 90:
        return "A"
    if grade >= 82:
        return "B"
    if grade >= 74:
        return "C"
    if grade >= 64:
        return "D"
    if grade >= 60:
        return "E"
    return "F"

def average(grades):
    return sum(grades) / len(grades)

def count_above(grades, limit):
    count = 0

    for grade in grades:
        if grade > limit:
            count += 1

    return count

def print_report(name, group, grades):
    avg = average(grades)

    print("\n--- Звіт про оцінки ---")
    print("Ім'я:", name)
    print("Група:", group)
    print("Оцінки:", grades)
    print(f"Середнє: {avg:.2f}")
    print("Буквена оцінка:", to_letter(avg))
    print("Найкращий бал:", max(grades))
    print("Найгірший бал:", min(grades))
    print("Оцінок вище середнього:", count_above(grades, avg))

def main():
    name = "Andriy"
    group = "IT-31"

    n = len(name)

```

```

    if n < 3:
        n = 3

    grades = []

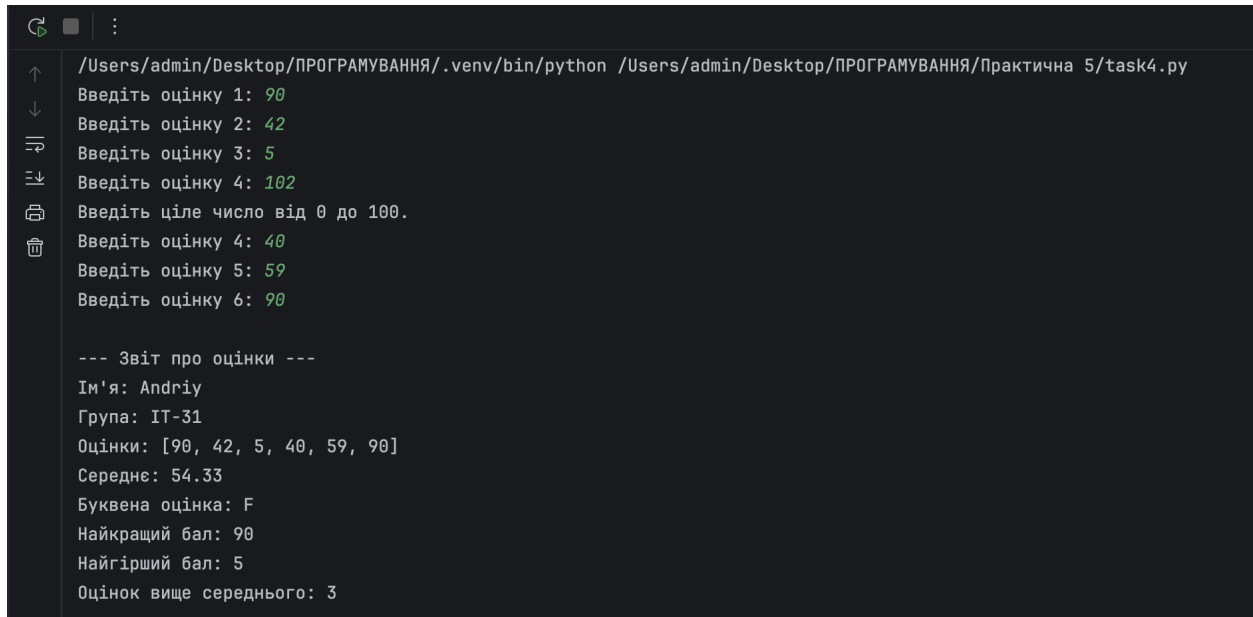
    for i in range(n):
        grade = read_grade(f"Введіть оцінку {i + 1}: ")
        grades.append(grade)

    print_report(name, group, grades)

main()

```

Результат:



```

/Users/admin/Desktop/ПРОГРАМУВАННЯ/.venv/bin/python /Users/admin/Desktop/ПРОГРАМУВАННЯ/Практична 5/task4.py
Введіть оцінку 1: 90
Введіть оцінку 2: 42
Введіть оцінку 3: 5
Введіть оцінку 4: 102
Введіть ціле число від 0 до 100.
Введіть оцінку 4: 40
Введіть оцінку 5: 59
Введіть оцінку 6: 90

--- Звіт про оцінки ---
Ім'я: Andriy
Група: IT-31
Оцінки: [90, 42, 5, 40, 59, 90]
Середнє: 54.33
Буквена оцінка: F
Найкращий бал: 90
Найгірший бал: 5
Оцінок вище середнього: 3

```

Відповідь на питання:

Порівняно з програмою з практичної роботи 4, використання функцій має три основні переваги: код стає більш структурованим і зрозумілим, окремі частини програми можна повторно використовувати, а змінювати або перевіряти окрему функцію значно простіше, не змінюючи всю програму.